

HTTP(S)

Contents

1	¿Qué es HTTP?	2
2	Estructura de un mensaje HTTP	2
3	Métodos	2
3.1	GET	2
3.2	HEAD	3
3.3	POST	3
3.4	PUT	3
3.5	DELETE	3
3.6	CONNECT	3
3.7	OPTIONS	3
3.8	TRACE	3
3.9	PATCH	3
3.10	QUERY	3
4	Ejemplo de Dialogo	3
5	Status Codes	4
5.1	Status Codes Comunes	4
6	HTTPS	4
6.1	Certificado TLS/SSL	5

1 ¿Qué es HTTP?

HTTP (Hypertext Transfer Protocol) es el protocolo encargado de la transferencia de información a través de archivos (XML, HTML, etc.). HTTP define la sintaxis y la semántica que utilizan los elementos de software de la arquitectura web (clientes, servidores, proxies) para comunicarse. **El puerto estandar de HTTP es el 80.**

Es un protocolo orientado a transacciones y sigue el esquema petición-respuesta entre un cliente y un servidor. El **cliente** realiza una petición enviando un mensaje, con cierto formato al servidor. El **servidor** le envía un mensaje de respuesta.

2 Estructura de un mensaje HTTP

Los mensajes HTTP son en texto plano, lo que lo hace más legible y fácil de depurar. Los mensajes tienen la siguiente estructura:

- Línea inicial (termina con retorno de carro y un salto de línea) con:
 - * Para las **peticiones**: la acción requerida por el servidor (método de petición) seguido de la URL del recurso y la versión de HTTP que soporta el cliente.
 - * Para **respuestas**: la versión del HTTP usado seguido del código de respuesta (que indica qué ha pasado con la petición seguido de la URL del recurso) y de la frase asociada a dicho retorno.
- Las **cabeceras** del mensaje que terminan con una línea en blanco. Son metadatos. Estas cabeceras le dan gran flexibilidad al protocolo.
- Cuerpo del mensaje (*Opcional*). Su presencia depende de la línea anterior del mensaje y del tipo de recurso al que hace referencia la URL. Típicamente tiene los datos que se intercambian cliente y servidor. Por ejemplo, para una petición podría contener ciertos datos que se quieren enviar al servidor para que los procese. Para una respuesta podría incluir los datos que el cliente ha solicitado.

3 Métodos

HTTP define una **serie predefinida de métodos** de petición para indicar la acción que se desea realizar sobre el **recurso** identificado. Que este recurso sea preexistente (datos ya almacenados) o se genere de forma dinámica depende de la implementación del servidor. A menudo, el recurso corresponde a un archivo o a la salida de un ejecutable que reside en el servidor.

Un método puede ser catalogado como **seguro, idempotente o cacheable**:

- Método Seguro: Si una petición con ese método no tiene un efecto intencional en el servidor (**solo lectura**).
- Método idempotente: Si múltiples peticiones con ese método tienen el mismo efecto que una sola petición. Los métodos seguros son trivialmente idempotentes pues no deberían representar actualizaciones para el servidor.
- Método Cacheable: Si se pueden almacenar sus resultados para su reutilización.

Método	Petición con Carga Util	Respuesta con Carga Util	Safe	Idempotente	Cacheable
GET	Opcional	Sí	Sí	Sí	Sí
HEAD	Opcional	No	Sí	Sí	Sí
POST	Sí	Sí	No	No	Sí
PUT	Sí	Sí	No	Sí	No
DELETE	Opcional	Sí	No	Sí	No
CONNECT	Opcional	Sí	No	No	No
OPTIONS	Opcional	Sí	Sí	Sí	No
TRACE	No	Sí	Sí	Sí	No
PATCH	Sí	Sí	No	No	No
QUERY			Sí	Sí	Sí

3.1 GET

El método **GET** solicita que el recurso de destino transfiera una representación de su estado. Las solicitudes que usan GET solo deben recuperar datos y no deben tener ningún otro efecto. *Traeme el recurso.*

3.2 HEAD

El método **HEAD** solicita que el recurso de destino transfiera una representación de su estado, al igual que el método GET, pero sin los datos de representación contenidos en el cuerpo de la respuesta. Esto es útil para poder recuperar los metadatos de las cabeceras de la respuesta, sin tener que transportar el contenido completo. *Traeme SOLO la etiqueta del recurso.*

3.3 POST

El método **POST** solicita que el recurso de destino procese la representación contenida en la petición de acuerdo con la semántica del propio recurso. **A nivel semántico está orientado a crear un nuevo recurso.** *Crea el recurso que con los datos que te voy a pasar.*

3.4 PUT

El método **PUT** envía datos al servidor, pero a diferencia del método POST la URI de la línea de petición no hace referencia al recurso que los procesará, sino que identifica a los propios datos. PUT está **más orientado a la actualización** de los mismos (aunque también podría crearlos). *Actualiza (o crea) el recurso con los siguientes datos.*

3.5 DELETE

Borra el recurso.

3.6 CONNECT

Es una solicitud que establece un **Túnel TCP/IP** mediante un proxy HTTP, con el fin de facilitar comunicación mediante **HTTPS**. *Solicito establecer conexión contigo.*

3.7 OPTIONS

Se usa para **mostrar las opciones presentes durante una conexión para un URL específico**, con el fin de que el cliente sea capaz de determinar qué métodos y/o cabeceras HTTP están permitidas durante una conexión. *¿Qué solicitudes te puedo hacer respecto a este recurso?*

3.8 TRACE

El método **TRACE** Solicita al servidor que introduzca en la respuesta todos los datos que reciba en el mensaje de petición. Se utiliza con fines de depuración y diagnóstico ya que el cliente puede ver lo que llega al servidor y de esta forma ver todo lo que añaden al mensaje los servidores intermedios *Dime TODO lo que los servidores intermedios añaden a mi mensaje.*

3.9 PATCH

Su función es la misma que PUT, el cual **sobrescribe completamente un recurso**. Se utiliza para actualizar, de manera parcial una o varias partes. Está orientado también para el uso con proxy. *Actualiza el recurso.*

3.10 QUERY

Similar a GET, **QUERY** solicita un recurso con la diferencia de poder solicitarlo mediante estructuras de consulta (queries) como los son **JSON** o **SQL**. Esto elimina la limitación del tamaño de URLs o la no idempotencia de la semántica de POST. *Busca el recurso utilizando este JSON o esta consulta SQL.*

4 Ejemplo de Dialogo

- Cliente:

```
GET /index.html HTTP/1.1
Host: www.example.com
Referer: www.google.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/
Connection: keep-alive
[Linea en blanco]
```

- Servidor:

```
HTTP/1.1 200 OK
Date: Fri, 31 Dec 2003 23:59:59 GMT
Content-Type: text/html
Content-Length: 1221
```

```
<html lang="eo">
<head>
<meta charset="utf-8">
<title>Titulo del sitio</title>
</head>
<body>
<h1>Pagina principal de tuHost</h1>
(Contenido)
.
.
.
</body>
</html>
```

Para ver un **ejemplo de dialogo de cada uno de los métodos HTTP**, desde un shell linux:

```
curl -X [Metodo] https://example.com/resource (-d '{"key": "value"}')
```

5 Status Codes

Un **STATUS CODE** es un número que indica que ha pasado con la petición. El resto del contenido de la respuesta dependerá del valor de este código.

El número de los códigos están elegidos de tal forma que según si pertenece a una centena u otra se pueda identificar el tipo de respuesta que ha dado el servidor:

- Formato **1XX: LOGS**. Respuestas informativas. Indica que la petición ha sido recibida y se está procesando.
- Formato **2XX: Respuestas correctas**. Indica que la petición ha sido procesada correctamente.
- Formato **3XX: Respuestas de redirección**. Indica que el cliente necesita realizar más acciones para finalizar la petición. — *Lo que buscas ya no está aquí.*
- Formato **4XX: Errores del lado del cliente**. Indica que ha habido un error al procesar la petición del cliente pues algo fue mal de su lado.
- Formato **5XX: Errores del lado del servidor**. La petición llegó, pero no se pudo procesar debido a un fallo del lado del servidor.

5.1 Status Codes Comunes

Código	Significado	Descripción
200	SUCCESS	Petición Exitosa
301	REDIRECT	El cliente será redirigido a otro dominio
400	BAD REQUEST	
401	UNAUTHORIZED	El cliente no ha sido autenticado
403	FORBIDDEN	El cliente sí fue autenticado, más no tiene acceso al recurso.
404	NOT FOUND	El recurso solicitado no pudo ser encontrado.
500	INTERNAL SERVER ERROR	Problemas del lado del servidor.
503	SERVICE UNAVAILABLE	Servidor no disponible.

6 HTTPS

El **Hypertext Transfer Protocol Secure** es la versión segura de HTTP. Como se menciona anteriormente, los mensajes HTTP son meramente texto plano, lo que los vuelve inseguros, pues cualquier persona que intercepte el mensaje podrá ver y entender su contenido. **El puerto estandar de HTTP es el 443.**

HTTPS resuelve esto al agregar un protocolo de encriptación llamado **Transport Layer Security (TLS)**, anteriormente conocido como **Secure Sockets Layer (SSL)**, que utiliza una **infraestructura de clave pública asimétrica**, lo que indica que este tipo de sistema utiliza dos claves diferentes para encriptar las comunicaciones.

- Clave Privada: Es la clave con la que cuenta el propietario de un sitio Web. Se ubica en un servidor web y se utiliza para descryptar la información encriptada por la clave publica.
- Clave Publica: Esta es utilizada por todos los clientes que quieran interactuar con el servidor de forma segura. Sirve para encriptar las peticiones, mas no para descryptarlas.

6.1 Certificado TLS/SSL

Para que un sitio Web pueda usar HTTPS, este necesita tener un **Certificado TLS/SSL**, los cuales pueden ser otorgados por proveedores de alojamiento de sitios. Estos certificados sirven para validar que un sitio web es quien dice ser. Se trata de un archivo almacenado en el servidor original de un sitio web.

El certificado cuenta con las claves publica y privada necesarias para encriptación TLS/SSL. De ese modo, cuando un usuario quiere entablar conversación con un servidor, este le puede pasar la clave publica necesaria para iniciar la comunicación encriptada.

Para obtener información detallada del certificado TLS/SSL, puedes usar el siguiente comando:

```
echo | openssl s_client -connect dominio.com:443 -servername dominio.com 2>/dev/null |  
openssl x509 -noout -text
```